

net-09 - Troubleshooting - OVN/OVS

Contexto

Neste treinamento iremos verificar a saúde dos vários componentes da solução, passando pelo Neutron, OVN Central e chegando nos Chassis. Teremos um exemplo de troubleshooting que usará o conhecimento adquirido nos treinamentos anteriores, que exigirá obter informações do Openstack, entendimento dos Logical Flows do OVN e seguirá o caminho dos pacotes no dataplane.

Os principais tópicos desse treinamento serão:

- Verificando o status dos agentes de network do Neutron e logs
- Analisando a saúde do cluster do OVN Central
- Validando os componentes do Chassi
- Troubleshooting de conectividade

 Esse treinamento utiliza o setup do treinamento 07

Verificando o status dos agentes de network do Neutron e logs

Neste exercício vamos avaliar os agentes de network do Neutron que são classificados em 3 tipos:

- **OVN Metadata agent** - É o agente de metadata que executa nos Chassis do tipo Compute Nodes
- **OVN Controller agent** - Agente do ovn-controller do tipo Chassi que executa nos Compute Nodes
- **OVN Controller Gateway agent** - Agente do ovn-controller do tipo Chassi Gateway que executa nos Gateway Nodes

A partir do Openstack, podemos avaliar o status dos agentes do ponto de vista do Neutron:

```
1 $ source openrc admin
2 $ openstack network agent list
3 +-----+-----+-----+-----+-----+
4 | ID | Agent Type | Host | Availability Zone | Alive |
5 +-----+-----+-----+-----+-----+
6 | fabc045a-d8e4-565b-a958-55eba692c152 | OVN Metadata agent | ovn-sandbox1 | | :-)
```

 O status : -) significa que o agente está UP

Faça o teste de parar o serviço do ovn-controller no Gateway Node:

```
1 # systemctl stop ovn-controller
```

Aguarde por alguns instantes(60 segundos) e avalie novamente o status dos agentes de network a partir do Neutron:

```
1 $ openstack network agent list
2 +-----+-----+-----+-----+-----+
3 | ID | Agent Type | Host | Availability Zone | Alive |
4 +-----+-----+-----+-----+-----+
5 | fabc045a-d8e4-565b-a958-55eba692c152 | OVN Metadata agent | ovn-sandbox1 | | :-)
```

 O status XXX significa que o agente está DOWN

Inicie novamente o serviço do ovn-controller no Gateway Node e verifique o status dos agentes.

Do ponto de vista do OVN Southbound database, podemos avaliar os Chassis registrados:

Até o momento, o OVN Central não possui um meio simples de verificar quais Chassis estão conectados no OVN Southbound, isso deve ser implementado em algum momento no futuro. Uma forma de avaliar a conectividade entre OVN Central e o Chassi, seria verificar o valor do campo **nb_cfg** da table Chassis_Private. Alias, esse é o método que o Neutron utiliza para determinar o status dos agentes do OVN Controller.

```

1 # ovn-sbctl list Chassis_Private
2 _uuid           : 5e9d4284-dd32-4721-955b-641f53051b45
3 chassis         : 5ffbc392-87b9-4e97-b6fc-1f9d8f71e5e6
4 external_ids    : {}
5 name            : "0727afaf-5048-4624-a898-1181367f0233"
6 nb_cfg          : 41
7 nb_cfg_timestamp : 1655486245700
8
9 _uuid           : e081c8fd-60ff-4eeb-b5c9-3998f19d2f08
10 chassis         : b283613e-8805-4bc0-a757-3c4897f9349f
11 external_ids    : {"neutron:ovn-metadata-id"="fabcd045a-d8e4-565b-a958-55eba692c152", "neutron:ovn-metadata-s
12 name            : "cab023cb-ba95-4ec3-aa9b-61c2267833cd"
13 nb_cfg          : 41
14 nb_cfg_timestamp : 1655486234372

```

Nesse momento, você pode parar o serviço do ovn-controller no Gateway Node:

Aguarde por alguns instantes(60 segundos) e avalie novamente o status dos agentes de network a partir do Neutron:

```
1 $ openstack network agent list
2 +-----
```

3	ID	Agent Type	Host	Availability Zone	Alive
4	+-----+-----+-----+-----+-----+				
5	fab045a-d8e4-565b-a958-55eba692c152	OVN Metadata agent	ovn-sandbox1		:-)
6	cab023cb-ba95-4ec3-aa9b-61c2267833cd	OVN Controller agent	ovn-sandbox1		:-)
7	0727afaf-5048-4624-a898-1181367f0233	OVN Controller Gateway agent	ovn-sandbox2		XXX
8	+-----+-----+-----+-----+-----+				

Nos logs do Neutron, é possível verificar o evento que originou a mudança do status do Chassi:

```
1 # journalctl -f | grep neutron
2 Jun 17 17:27:21 ovn-sandbox1 neutron-server[524307]: DEBUG neutron.plugins.ml2.drivers.ovn.mech_driver.ovsdb.ovsd
```

Execute novamente o comando abaixo no OVN Central para listar os Chassis e verifique o valor do campo **nb_cfg**:

```
1 # ovn-sbctl list Chassis_Private
2 _uuid          : 5e9d4284-dd32-4721-955b-641f53051b45
3 chassis        : 5ffbc392-87b9-4e97-b6fc-1f9d8f71e5e6
4 external_ids    : {}
5 name           : "0727afaf-5048-4624-a898-1181367f0233"
6 nb_cfg         : 45
7 nb_cfg_timestamp : 1655486748356
8
9 _uuid          : e081c8fd-60ff-4eeb-b5c9-3998f19d2f08
10 chassis        : b283613e-8805-4bc0-a757-3c4897f9349f
11 external_ids    : {"neutron:ovn-metadata-id"="fab045a-d8e4-565b-a958-55eba692c152", "neutron:ovn-metadata-s
12 name           : "cab023cb-ba95-4ec3-aa9b-61c2267833cd"
13 nb_cfg         : 47
14 nb_cfg_timestamp : 1655486841105
```

Conforme o output acima, o Chassi do Gateway Node possui o valor de **45** no campo **nb_cfg** e o Chassi do Compute Node possui o valor **47**. A diferença dos valores mostra que o ovn-controller do Gateway Node não está reportando nenhum status pelo motivo que o serviço foi parado.

Inicialize novamente o serviço do ovn-controller no Gateway Node:

```
1 # systemctl start ovn-controller
```

Execute novamente o comando abaixo no OVN Central para listar os Chassis e verifique o valor do campo **nb_cfg**:

```
1 # ovn-sbctl list Chassis_Private
2 _uuid          : 5e9d4284-dd32-4721-955b-641f53051b45
3 chassis        : 5ffbc392-87b9-4e97-b6fc-1f9d8f71e5e6
4 external_ids    : {}
5 name           : "0727afaf-5048-4624-a898-1181367f0233"
6 nb_cfg         : 47
7 nb_cfg_timestamp : 1655487205573
8
9 _uuid          : e081c8fd-60ff-4eeb-b5c9-3998f19d2f08
10 chassis        : b283613e-8805-4bc0-a757-3c4897f9349f
11 external_ids    : {"neutron:ovn-metadata-id"="fab045a-d8e4-565b-a958-55eba692c152", "neutron:ovn-metadata-s
12 name           : "cab023cb-ba95-4ec3-aa9b-61c2267833cd"
13 nb_cfg         : 47
14 nb_cfg_timestamp : 1655486841105
```

Conforme o output acima, o valor do campo **nb_cfg** de ambos os Chassis está igual.

 Se o serviço do ovn-controller estiver desconectado do OVN Southbound, qualquer alteração de rede não será sincronizada com o OVS. Se uma VM for criada nesse Chassi enquanto o serviço do ovn-controller estiver parado, os flows necessários para permitir o tráfego de rede da mesma não serão instalados no OVS e consequentemente o tráfego de rede será bloqueado.

Para avaliar os logs do Neutron, você pode executar o seguinte comando onde o neutron-server está em execução:

```
1 # journalctl -f | grep neutron
2 Jun 17 17:43:48 ovn-sandbox1 neutron-server[524307]: INFO neutron.wsgi [req-5129e2a8-e4e2-4099-ae81-8aad37d4c8f1
3 Jun 17 17:43:48 ovn-sandbox1 nova-compute[533622]: DEBUG nova.network.neutron [None req-8546637e-661c-48d7-8528-0
```

Analizando a saúde do cluster do OVN Central [↗](#)

Neste exercício, iremos analisar a saúde do cluster do OVN Central. Em nosso setup atual, não temos um cluster configurado e utilizaremos o ambiente de Staging.

O OVN Central é considerado como a Control Plane ou SDN Controller da solução do OVN e é composto pelos seguintes serviços:

- **OVN Northbound database**
- **ovn-northd**
- **OVN Southbound database**

No modelo de serviço com cluster, utiliza-se o [Raft](#) para gerenciar o cluster que permitirá a escrita e leitura das databases conforme o status dos membros do cluster. Em um cluster, ocorrerá a eleição do **Leader** que permitirá a escrita/leitura em suas databases locais enquanto os **Followers** sincronizam as alterações a partir do Leader e permitem apenas a leitura local das databases. Em uma eleição os seguintes status são esperados:

- **Follower** - Esse é o primeiro estado após o node entrar no cluster
- **Candidate** - Quando não existe um Leader no cluster, se inicia a eleição e o estado muda para Candidate
- **Leader** - Se receber a maioria dos votos, o estado passa para Leader e o restante dos nodes mudam seu estado para Follower

i O Leader é definido para o Northbound e também para o Southbound

Abaixo temos o output dos nodes de cluster do OVN Central (ambiente Staging):

OVN Central - node 1

```
1 # ovs-appctl -t /var/run/ovn/ovnnb_db.ctl cluster/status OVN_Northbound
2 a6f9
3 Name: OVN_Northbound
4 Cluster ID: 3c82 (3c825fc0-ec15-4e4e-80df-4c3c2a54ccbf)
5 Server ID: a6f9 (a6f9ca7b-1cb3-4aab-b114-2bcc1fb0e71b)
6 Address: tcp:10.24.36.25:6643
7 Status: cluster member
8 Role: follower
9 Term: 68
10 Leader: c3f7
11 Vote: c3f7
12
13 Election timer: 10000
14 Log: [366554, 366555]
15 Entries not yet committed: 0
16 Entries not yet applied: 0
17 Connections: <-ae3e ->ae3e <-c3f7 ->c3f7
18 Servers:
19     c3f7 (c3f7 at tcp:10.24.36.138:6643)
20     ae3e (ae3e at tcp:10.24.36.73:6643)
21     a6f9 (a6f9 at tcp:10.24.36.25:6643) (self)
22
23 # ovs-appctl -t /var/run/ovn/ovnsb_db.ctl cluster/status OVN_Southbound
24 990d
25 Name: OVN_Southbound
26 Cluster ID: 28c0 (28c0509f-78d1-4e40-b18f-9d6efab0b0f7)
```

```

27 Server ID: 990d (990d9df5-2d67-455f-aa90-00e8fef82354)
28 Address: tcp:10.24.36.25:6644
29 Status: cluster member
30 Role: follower
31 Term: 1626
32 Leader: 7a45
33 Vote: 7a45
34
35 Election timer: 10000
36 Log: [225530, 225531]
37 Entries not yet committed: 0
38 Entries not yet applied: 0
39 Connections: <-e873 ->e873 <-7a45 ->7a45
40 Servers:
41     990d (990d at tcp:10.24.36.25:6644) (self)
42     7a45 (7a45 at tcp:10.24.36.138:6644)
43     e873 (e873 at tcp:10.24.36.73:6644)

```

No output acima, podemos determinar:

- a role de ambos os serviços (Northbound e Southbound) está como **follower**
- o Election timer do cluster é de 10 segundos
- temos 3 nodes no cluster

OVN Central - node 2

```

1 # ovs-appctl -t /var/run/ovn/ovnnb_dbctl cluster/status OVN_Northbound
2 ae3e
3 Name: OVN_Northbound
4 Cluster ID: 3c82 (3c825fc0-ec15-4e4e-80df-4c3c2a54ccbf)
5 Server ID: ae3e (ae3e830e-8f7d-47c2-bc93-47fbc6b23673)
6 Address: tcp:10.24.36.73:6643
7 Status: cluster member
8 Role: follower
9 Term: 68
10 Leader: c3f7
11 Vote: c3f7
12
13 Election timer: 10000
14 Log: [366554, 366555]
15 Entries not yet committed: 0
16 Entries not yet applied: 0
17 Connections: ->0000 <-a6f9 <-c3f7 ->c3f7
18 Servers:
19     c3f7 (c3f7 at tcp:10.24.36.138:6643)
20     ae3e (ae3e at tcp:10.24.36.73:6643) (self)
21     a6f9 (a6f9 at tcp:10.24.36.25:6643)
22
23 # ovs-appctl -t /var/run/ovn/ovnsb_dbctl cluster/status OVN_Southbound
24 e873
25 Name: OVN_Southbound
26 Cluster ID: 28c0 (28c0509f-78d1-4e40-b18f-9d6efab0b0f7)
27 Server ID: e873 (e873f4cb-3ba0-4637-b64a-56e304c42074)
28 Address: tcp:10.24.36.73:6644
29 Status: cluster member
30 Role: follower
31 Term: 1626
32 Leader: 7a45
33 Vote: 7a45

```

```

34
35 Election timer: 10000
36 Log: [225530, 225531]
37 Entries not yet committed: 0
38 Entries not yet applied: 0
39 Connections: ->0000 <-990d <-7a45 ->7a45
40 Servers:
41     990d (990d at tcp:10.24.36.25:6644)
42     7a45 (7a45 at tcp:10.24.36.138:6644)
43     e873 (e873 at tcp:10.24.36.73:6644) (self)
44

```

No output acima, podemos determinar:

- a role de ambos os serviços (Northbound e Southbound) está como **follower**
- o Election timer do cluster é de 10 segundos
- temos 3 nodes no cluster

OVN Central - node 3

```

1 # ovs-appctl -t /var/run/ovn/ovnnb_dbctl cluster/status OVN_Northbound
2 c3f7
3 Name: OVN_Northbound
4 Cluster ID: 3c82 (3c825fc0-ec15-4e4e-80df-4c3c2a54ccbf)
5 Server ID: c3f7 (c3f7cf40-1a89-4df1-bd0a-1f2e842a588e)
6 Address: tcp:10.24.36.138:6643
7 Status: cluster member
8 Role: leader
9 Term: 68
10 Leader: self
11 Vote: self
12
13 Election timer: 10000
14 Log: [366553, 366555]
15 Entries not yet committed: 0
16 Entries not yet applied: 0
17 Connections: ->0000 ->ae3e <-a6f9 <-ae3e
18 Servers:
19     c3f7 (c3f7 at tcp:10.24.36.138:6643) (self) next_index=366554 match_index=366554
20     ae3e (ae3e at tcp:10.24.36.73:6643) next_index=366555 match_index=366554
21     a6f9 (a6f9 at tcp:10.24.36.25:6643) next_index=366555 match_index=366554
22
23 # ovs-appctl -t /var/run/ovn/ovnsb_dbctl cluster/status OVN_Southbound
24 7a45
25 Name: OVN_Southbound
26 Cluster ID: 28c0 (28c0509f-78d1-4e40-b18f-9d6efab0b0f7)
27 Server ID: 7a45 (7a45e9c7-4d8a-4c7d-a36c-271c858d854e)
28 Address: tcp:10.24.36.138:6644
29 Status: cluster member
30 Role: leader
31 Term: 1626
32 Leader: self
33 Vote: self
34
35 Election timer: 10000
36 Log: [225530, 225531]
37 Entries not yet committed: 0
38 Entries not yet applied: 0
39 Connections: ->0000 ->e873 <-990d <-e873

```

```

40 Servers:
41     990d (990d at tcp:10.24.36.25:6644) next_index=225531 match_index=225530
42     7a45 (7a45 at tcp:10.24.36.138:6644) (self) next_index=225530 match_index=225530
43     e873 (e873 at tcp:10.24.36.73:6644) next_index=225531 match_index=225530
44

```

No output acima, podemos determinar:

- a role de ambos os serviços (Northbound e Southbound) está como **leader**
- o Election timer do cluster é de 10 segundos
- temos 3 nodes no cluster

Em um cluster de OVN Central, caso se queira executar um comando em um node que não seja o leader, utilize o parâmetro **--no-leader-only** conforme abaixo:

```

1 # ovn-sbctl --no-leader-only list Chassis

```

Os arquivos de log de eleição do leader do OVN Central e de eventos do Northbound/Southbound são:

- **OVN Northbound** - /var/log/ovn/ovsdb-server-nb.log
- **OVN Southbound** - /var/log/ovn/ovsdb-server-sb.log

Arquivo de log do **ovn-northd**:

- /var/log/ovn/ovn-northd.log

Em algumas situações, a releição do leader do cluster pode ocorrer:

- o node leader ou o serviço do OVN Central ficam offline - ocorrerá uma releição do Northbound/Southbound com os 2 nodes restantes
- snapshot das databases Northbound e Southbound - O snapshot não é configurável e o intervalo de tempo depende da quantidade de objetos nas databases. Antes de iniciar o snapshot, o leader irá solicitar uma nova releição e 2 nodes irão decidir o novo leader. O antigo leader se tornará follower.

Os serviços do OVN Northbound, OVN Southbound e ovn-northd podem ser verificados com o seguinte comando:

```

1 # systemctl status ovn-central
2 • ovn-central.service - Open Virtual Network central components
3     Loaded: loaded (/lib/systemd/system/ovn-central.service; enabled; vendor preset: enabled)
4     Active: active (exited) since Mon 2022-06-06 21:06:17 UTC; 1min 8s ago
5     Process: 3115 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
6     Main PID: 3115 (code=exited, status=0/SUCCESS)
7
8 Jun 06 21:06:16 ovn-sandbox1 systemd[1]: Starting Open Virtual Network central components...
9 Jun 06 21:06:17 ovn-sandbox1 systemd[1]: Finished Open Virtual Network central components.

```

Validando os componentes do Chassi

Neste exercício, faremos a validação dos componentes que executam no nível do Chassi.

No Chassi, os componentes locais que devem estar em execução são:

- **ovn-controller**
- **OVS**
- **OVN Metadata agent** (somente em Compute Nodes)

Para avaliar o status dos componentes, execute os seguintes comandos:

```

1 # systemctl status ovn-controller
2 • ovn-controller.service - Open Virtual Network host control daemon
3     Loaded: loaded (/lib/systemd/system/ovn-controller.service; static; vendor preset: enabled)

```

```

4      Active: active (running) since Thu 2022-06-09 12:03:17 UTC; 1 weeks 1 days ago
5      Main PID: 523083 (ovn-controller)
6      Tasks: 4 (limit: 9556)
7      Memory: 7.1M
8      CGroup: /system.slice/ovn-controller.service
9          └─523083 ovn-controller unix:/var/run/openvswitch/db.sock -vconsole:emer -vsyslog:err -vfile:info -
10
11 # systemctl status openvswitch-switch
12 • openvswitch-switch.service - Open vSwitch
13     Loaded: loaded (/lib/systemd/system/openvswitch-switch.service; enabled; vendor preset: enabled)
14     Active: active (exited) since Fri 2022-06-17 19:07:20 UTC; 1s ago
15     Process: 1454817 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
16     Main PID: 1454817 (code=exited, status=0/SUCCESS)
17
18 Jun 17 19:07:20 ovn-sandbox1 systemd[1]: Starting Open vSwitch...
19 Jun 17 19:07:20 ovn-sandbox1 systemd[1]: Finished Open vSwitch.
20
21 # systemctl status neutron-ovn-metadata-agent
22 • neutron-ovn-metadata-agent.service - neutron-ovn-metadata-agent service
23     Loaded: loaded (/etc/systemd/system/neutron-ovn-metadata-agent.service; enabled; vendor preset: enabled)
24     Active: active (running) since Tue 2022-05-31 21:25:27 UTC; 2 weeks 2 days ago
25     Main PID: 3399188 (/usr/bin/python)
26     Tasks: 3068 (limit: 629145)
27     Memory: 3.2G
28     CPU: 1d 17h 59min 16.867s
29     CGroup: /neutron.slice/neutron-ovn-metadata-agent.service

```

i Em um setup com Devstack, o serviço do OVN metadata se chama **devstack@q-ovn-metadata-agent.service**

O mau funcionamento desses serviços pode resultar em problemas como os descritos abaixo:

- Em um cenário em que o serviço do ovn-controller estiver parado ou se o OVN Central tiver algum problema no cluster, ações como criar uma nova VM em um Compute Node pode resultar em um isolamento da VM. Isso ocorre devido ao OVS depender do serviço do ovn-controller para programar os Openflow flows
- O serviço do ovn-controller pode ser reiniciado normalmente sem afetar o tráfego de rede das VMs do dataplane, porém ao reiniciar o serviço do OVS, haverá a perda de pacotes das VMs durante a instalação dos Openflow flows
- O agente do OVN metadata executa funções muito importantes após a criação de uma VM. As principais configurações são como definir o hostname, configurar a chave pública de SSH e dentre outras personalizações do sistema operacional via cloud-init. Caso o mesmo venha sofrer uma reconexão com o OVN Southbound no momento da execução de alguma dessas funções, a VM poderá ter algum problema.

Os arquivos de log do Chassi que podem ser verificados durante um troubleshooting são os abaixo:

- **ovn-controller** - /var/log/ovn/ovn-controller.log
- **ovsdb-server** - /var/log/openvswitch/ovsdb-server.log
- **ovs-vswitchd** - /var/log/openvswitch/ovs-vswitchd.log
- **OVN Metadata** - /var/log/syslog

No nível de dataplane, antes do tráfego de rede ser tunelado com Geneve ou mesmo ser enviado localmente no mesmo Chassi, é possível monitorar o tráfego de rede via interface de rede tap de uma VM. Para que isso seja possível, verifique primeiramente em qual Compute Node que a VM está em execução:

```

1 $ openstack server list
2 +-----+-----+-----+-----+
3 | ID | Name | Status | Networks |
4 +-----+-----+-----+-----+
5 | bd4d1375-a5b6-4d01-aa6b-8c9ebbe08dd1 | vm10 | ACTIVE | private2=10.0.0.67

```

```

6 | 3305e19e-41c5-441b-b9fb-99fd0fe05d3a | vm2 | ACTIVE | private=10.0.0.7, fd42:4193:f777:0:f816:3eff:fe5f:333f
7 | dac8d874-1aa5-494c-88a6-234fd1575201 | vm1 | ACTIVE | private=10.0.0.42, 172.24.4.96, fd42:4193:f777:0:f816:3
8 | +-----+-----+-----+-----+
9 | $ openstack server show dac8d874-1aa5-494c-88a6-234fd1575201 | grep OS-EXT-S
10 | OS-EXT-SRV-ATTR:host | ovn-sandbox1 |
11 | OS-EXT-SRV-ATTR:hypervisor_hostname | ovn-sandbox1 |
12 | OS-EXT-SRV-ATTR:instance_name | instance-00000002 |
13 | OS-EXT-STS:power_state | Running |
14 | OS-EXT-STS:task_state | None |
15 | OS-EXT-STS:vm_state | active |

```

Conforme o output acima, a VM1 está executando no Compute Node `ovn-sandbox1` e o instance name é `instance-00000002`.

Com acesso ao respectivo Compute Node, execute:

```
1 # virsh domiflist instance-00000002
2 Interface      Type      Source      Model      MAC
3 -----
4 tap04a459b1-6e  ethernet  -           virtio      fa:16:3e:77:12:96
```

A interface de rede `tap04a459b1-6e` está conectada no OVS mas podemos verificar o tráfego de rede via `tcpdump`:

```

1 # tcpdump -nni tap04a459b1-6e
2 tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
3 listening on tap04a459b1-6e, link-type EN10MB (Ethernet), capture size 262144 bytes
4 20:10:05.315489 IP 10.0.0.42 > 172.24.4.1: ICMP echo request, id 49669, seq 9, length 64
5 20:10:05.316183 IP 172.24.4.1 > 10.0.0.42: ICMP echo reply, id 49669, seq 9, length 64
6 20:10:06.316421 IP 10.0.0.42 > 172.24.4.1: ICMP echo request, id 49669, seq 10, length 64
7 20:10:06.317263 IP 172.24.4.1 > 10.0.0.42: ICMP echo reply, id 49669, seq 10, length 64
8 ^C
9 4 packets captured
10 4 packets received by filter
11 0 packets dropped by kernel

```

No output acima, podemos verificar que existe um tráfego de rede de ICMP ativo.

Uma outra opção que temos é verificar os Physical Flows instalados no Datapath, mas teremos que ser específicos no filtro para não se confundir. Neste caso, pode-se usar o endereço IP, porém pode existir overlapping entre diferentes tenants e o MAC Address seria o mais assertivo:

```
1 # watch -n 2 "ovs-dpctl dump-flows | grep fa:16:3e:77:12:96"
2 recirc_id(0x9),in_port(4),ct_state(-new+est-rel-rpl-inv+trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=fa:16:
3 recirc_id(0),in_port(4),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e:8f:c6:8b),eth_type(0x0800),ipv4(src=10.0.0.42,prot
4 recirc_id(0),in_port(4),ct_state(-new+est-rel-rpl-inv+trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e
5 recirc_id(0xa),tunnel(tun_id=0x1,src=10.0.10.200,dst=10.0.10.100,geneve({}{}),flags(-df+csum+key)),in_port(3),ct
```

O resultado do output nos mostra que o tráfego de rede está sendo enviado para o Chassi 10.0.10.200 via túnel Geneve.

Troubleshooting de conectividade

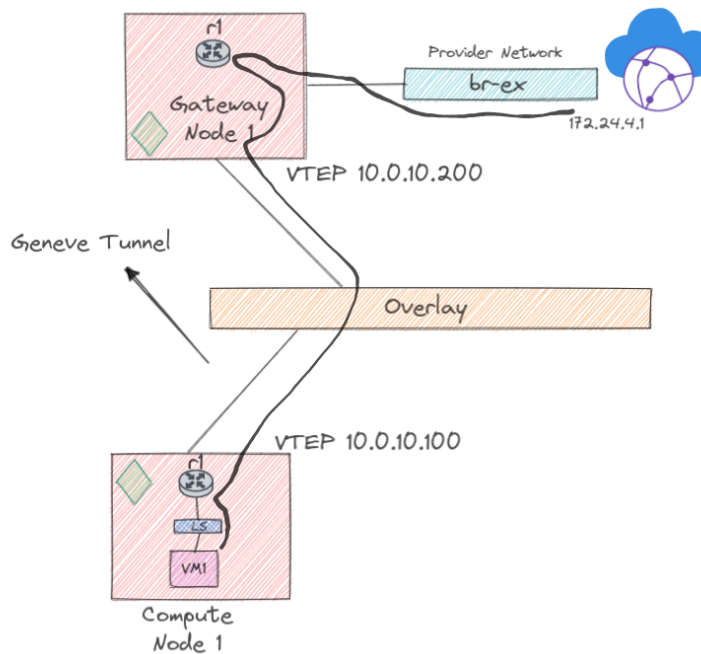
Neste exercício, teremos que usar o conhecimento adquirido anteriormente para executar um troubleshooting de conectividade norte-sul. Estaremos usando o mesmo cenário do treinamento anterior (08-Packet Walkthrough).

Durante qualquer troubleshooting, tente reunir a maior quantidade de informações possíveis para ajudar a determinar padrões que ajudarão a encontrar a causa raiz do problema:

- o problema ocorre em apenas uma VM ou em várias?
- é específico em um tenant do Openstack ou afeta vários?

- afeta o tráfego de rede local em um único Compute Node ou também é notado no tráfego de rede leste-oeste? Identifique os Compute Nodes afetados e verifique os componentes da solução
- avalie a conectividade norte-sul, Gateway Nodes e também a conectividade de rede do Underlay
- existe load de CPU anormal no Compute Node/Gateway Node? Perda de pacotes no nível de Underlay?
- a VM tem conectividade com seu gateway de rede?

Faremos um walkthrough para verificar os principais hops na comunicação norte-sul do seguinte cenário entre a VM1 e o endereço IP de externo 172.24.4.1:



Antes de mais nada, algo que ajuda a entender a topologia de rede do cliente é verificar via Openstack o que existe configurado:

```
1 $ openstack network list
2 $ openstack subnet show 229329b1-1627-49e2-bfb3-deacba7155f9
3 $ openstack port list --network 5bb5b9ad-7348-474f-9e14-66f34278e373
4 $ openstack router list
5 $ openstack router show d18829fb-2cf8-47b7-85d6-0e96caf2e90f
6 $ openstack server list
7 $ openstack server show dac8d874-1aa5-494c-88a6-234fd1575201
8 $ openstack security group list --project demo
9 $ openstack security group rule list fed05084-5a95-4054-87e3-794fca4f7075
```

Com o output acima, é possível determinar se algo como uma simples regra de Security Group está faltando.

Após determinar em qual Compute Node uma VM está em execução, execute os seguintes passos:

1- Com acesso ao respectivo Compute Node, execute:

```
1 # virsh domiflist instance-00000002
2 Interface      Type      Source    Model    MAC
3 -----
4 tap04a459b1-6e ethernet -         virtio   fa:16:3e:77:12:96
```

2 - A interface de rede `tap04a459b1-6e` está conectada no OVS mas podemos verificar o tráfego de rede via **tcpdump**:

```
1 # tcpdump -nni tap04a459b1-6e
2 tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
3 listening on tap04a459b1-6e, link-type EN10MB (Ethernet), capture size 262144 bytes
```

```

4  20:10:05.315489 IP 10.0.0.42 > 172.24.4.1: ICMP echo request, id 49669, seq 9, length 64
5  20:10:05.316183 IP 172.24.4.1 > 10.0.0.42: ICMP echo reply, id 49669, seq 9, length 64
6  20:10:06.316421 IP 10.0.0.42 > 172.24.4.1: ICMP echo request, id 49669, seq 10, length 64
7  20:10:06.317263 IP 172.24.4.1 > 10.0.0.42: ICMP echo reply, id 49669, seq 10, length 64
8  ^C
9  4 packets captured
10 4 packets received by filter
11 0 packets dropped by kernel

```

```
1 # watch -n 2 "ovs-dpctl dump-flows | grep fa:16:3e:77:12:96"
2 recirc_id(0x9),in_port(4),ct_state(-new+est-rel-rpl-inv+trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=fa:16:
3 recirc_id(0),in_port(4),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e:8f:c6:8b),eth_type(0x0800),ipv4(src=10.0.0.42,prot
4 recirc_id(0),in_port(4),ct_state(-new+est-rel-rpl-inv+trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e
5 recirc_id(0xa),tunnel(tun_id=0x1,src=10.0.10.200,dst=10.0.10.100,geneve({}{}),flags(-df+csum+key)),in_port(3),ct
```

4 - Ainda no Compute Node, confirme qual interface de rede do Compute Node possui o endereço IP de VTEP:

Verifique se o tráfego de rede está sendo tunelado com Geneve:

Conforme o output acima, o tráfego de rede está sendo enviado para o Chassi 10.0.10.200.

```
1 # ovs-vsctl list open_vswitch | grep ovn-encap-ip
2 external_ids      : {hostname=ovn-sandbox2, ovn-bridge=br-int, ovn-bridge-mappings="public:br-ex", ovn-cms-opti
3 # ip a | grep 10.0.10.200
4     inet 10.0.10.200/24 brd 10.0.10.255 scope global enp1s0.10
```

```
1 # tcpdump -nni enp1s0.10 geneve
2 tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
3 listening on enp1s0.10, link-type EN10MB (Ethernet), capture size 262144 bytes
4 21:03:32.747071 IP 10.0.10.100.18332 > 10.0.10.200.6081: Geneve, Flags [C], vni 0x2, options [8 bytes]: IP 10.0.
5 21:03:32.747301 IP 10.0.10.200.55963 > 10.0.10.100.6081: Geneve, Flags [C], vni 0x1, options [8 bytes]: IP 172.2
6 21:03:33.748150 IP 10.0.10.100.18332 > 10.0.10.200.6081: Geneve, Flags [C], vni 0x2, options [8 bytes]: IP 10.0.
```

```

7 21:03:33.748310 IP 10.0.10.200.55963 > 10.0.10.100.6081: Geneve, Flags [C], vni 0x1, options [8 bytes]: IP 172.2
8 21:03:34.749208 IP 10.0.10.100.18332 > 10.0.10.200.6081: Geneve, Flags [C], vni 0x2, options [8 bytes]: IP 10.0.
9 21:03:34.749364 IP 10.0.10.200.55963 > 10.0.10.100.6081: Geneve, Flags [C], vni 0x1, options [8 bytes]: IP 172.2
10 ^C
11 6 packets captured

```

Podemos concluir que o tráfego de rede tunelado com Geneve está chegando corretamente no Gateway Node.

6 - Verifique o Datapath para entendermos se o OVS instalou os flows corretamente:

```

1 # watch -n 2 "ovs-dpctl dump-flows | grep fa:16:3e:77:12:96"
2 recirc_id(0),in_port(3),ct_state(-new+est-rel+rp1-inv+trk),ct_label(0/0x1),eth(src=0e:d5:cd:6a:ae:43,dst=fa:16:3e

```

Aparentemente temos um flow ativo referente ao tráfego de rede de egress.

Podemos alterar o filtro, por exemplo verifique com o endereço do Floating IP da VM ou de SNAT do router se for o caso:

```

1 # ovs-dpctl dump-flows | grep 172.24.4.96
2 recirc_id(0),tunnel(tun_id=0x2,src=10.0.10.100,dst=10.0.10.200,geneve({class=0x102,type=0x80,len=4,0x10003/0x7fff
3 recirc_id(0),in_port(3),ct_state(-new+est-rel+rp1-inv+trk),ct_label(0/0x1),eth(src=0e:d5:cd:6a:ae:43,dst=fa:16:3e

```

O último passo é entender se o tráfego de rede sai do OVS e entra na bridge da network provider, primeiramente verifique qual o nome da bridge:

```

1 # ovs-vsctl list open_vswitch | grep ovn-bridge-mappings
2 external_ids      : {hostname=ovn-sandbox2, ovn-bridge=br-int, ovn-bridge-mappings="public:br-ex", ovn-cms-opti

```

Em nosso setup a bridge é a `br-ex`. Execute o tcpdump nessa interface de rede e analise o tráfego de rede:

```

1 # tcpdump -nni br-ex host 172.24.4.1
2 tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
3 listening on br-ex, link-type EN10MB (Ethernet), capture size 262144 bytes
4 21:13:02.371663 IP 172.24.4.96 > 172.24.4.1: ICMP echo request, id 49669, seq 3782, length 64
5 21:13:02.371743 IP 172.24.4.1 > 172.24.4.96: ICMP echo reply, id 49669, seq 3782, length 64
6 21:13:03.373075 IP 172.24.4.96 > 172.24.4.1: ICMP echo request, id 49669, seq 3783, length 64
7 21:13:03.373154 IP 172.24.4.1 > 172.24.4.96: ICMP echo reply, id 49669, seq 3783, length 64
8 ^C
9 4 packets captured

```

Como é possível verificar, o ICMP echo request é enviado e o echo reply é recebido.

O mapeamento dos MAC Addresses externos, que são aprendidos via ARP pode ser verificado no OVN Southbound:

```

1 # ovn-sbctl list Mac_Binding
2 _uuid          : f880e921-6546-4dd9-afb0-59b257b462ba
3 datapath       : 4f59f6dd-5745-4634-8cd4-52af21720b31
4 ip             : "172.24.4.1"
5 logical_port   : lrp-cf47f4af-5cfd-4293-8254-1573856157c1
6 mac           : "0e:d5:cd:6a:ae:43"

```

Conclusão

Neste treinamento verificamos a saúde dos vários componentes da solução passando pelo Neutron, OVN Central e chegando nos Chassis. Executamos um troubleshooting de conectividade norte-sul seguindo o caminho dos pacotes no dataplane.